

Homework 1

Yiyi Zhu

2026-04-25

Part 1:

1.1

(a)

```
library(dplyr)
```

Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

filter, lag

The following objects are masked from 'package:base':

intersect, setdiff, setequal, union

```
vectorized_zscore <- function(x) {  
  (x - mean(x, na.rm = TRUE)) / sd(x, na.rm = TRUE)  
}
```

```
loop_zscore <- function(x) {  
  n <- length(x)  
  res <- numeric(n)  
  m <- mean(x, na.rm = TRUE)
```

```

s <- sd(x, na.rm = TRUE)
for (i in 1:n) {
  res[i] <- (x[i] - m) / s
}
return(res)
}

```

###(b)

```

library(bench)
library(ggplot2)

bm_results <- bench::mark(
  vectorized = vectorized_zscore(iris$Sepal.Length),
  loop = loop_zscore(iris$Sepal.Length),
  iterations = 50
)

print(bm_results)

```

```

# A tibble: 2 x 13
  expression      min median `itr/sec` mem_alloc `gc/sec` n_itr n_gc total_time
  <bch:expr> <bch:tm> <bch:>      <dbl> <bch:byt>      <dbl> <int> <dbl>    <bch:tm>
1 vectorized   11.9us 12.9us   20488.   68.4KB         0    50     0     2.44ms
2 loop        22.8us 23.6us   41493.   52.5KB         0    50     0     1.21ms
# i 4 more variables: result <list>, memory <list>, time <list>, gc <list>

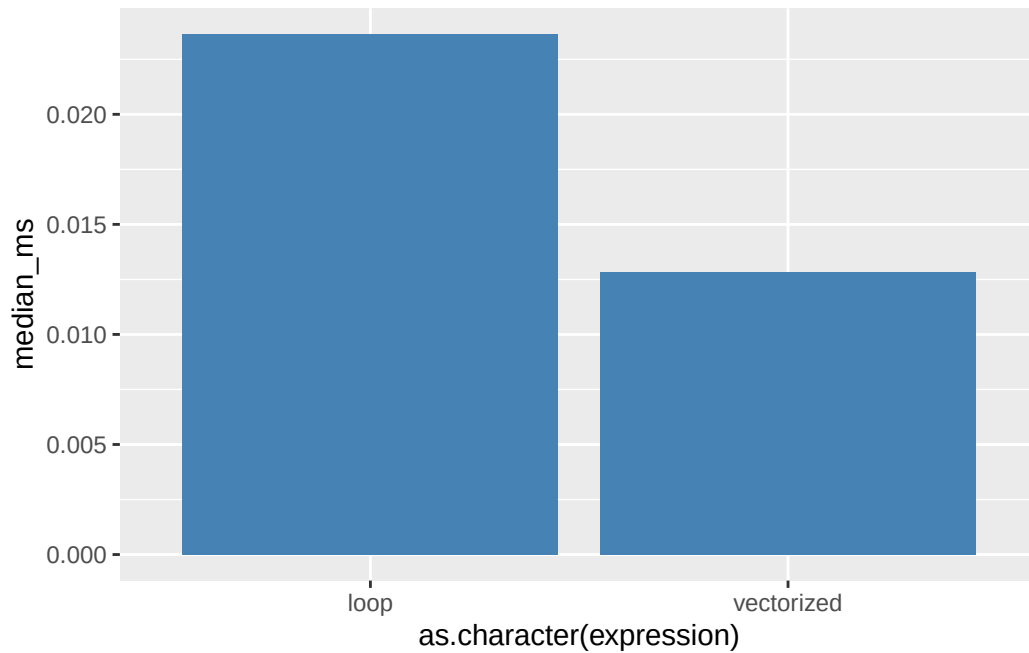
```

```

plot_data <- as.data.frame(bm_results) %>%
  mutate(median_ms = as.numeric(median) * 1000)

ggplot(plot_data, aes(x = as.character(expression), y = median_ms)) +
  geom_col(fill = "steelblue")

```



###(c)

```
library(dplyr)
library(knitr)

chick_stats <- ChickWeight %>%
  group_by(Chick) %>%
  mutate(

    Growth.Rate = (last(weight) - first(weight)) / (last(Time) - first(Time)),

    Weight.Gain.Percent = (last(weight) - first(weight)) / first(weight) * 100,

    Max_Weight_Each = max(weight, na.rm = TRUE)
  ) %>%
  slice(1) %>%
  ungroup()

diet_summary <- chick_stats %>%
  group_by(Diet) %>%
  summarise(
    Avg_Growth_Rate = mean(Growth.Rate, na.rm = TRUE),
```

```

    Avg_Gain_Percent = mean(Weight.Gain.Percent, na.rm = TRUE), # <--
    Mean_Max_Weight = mean(Max_Weight_Each, na.rm = TRUE)
  )

kable(diet_summary, digits = 2)

```

Diet	Avg_Growth_Rate	Avg_Gain_Percent	Mean_Max_Weight
1	5.45	276.95	157.95
2	8.29	431.73	215.60
3	10.93	560.79	271.90
4	9.05	461.37	229.90

1.2 Data Structures and Functional Programming

(a)

```
library(palmerpenguins)
```

Attaching package: 'palmerpenguins'

The following objects are masked from 'package:datasets':

penguins, penguins_raw

```

s_names <- unique(penguins$species)

penguin_species_list <- lapply(s_names, function(s) {

  df_subset <- penguins[penguins$species == s, ]

  attr(df_subset, "sample_size") <- nrow(df_subset)
  return(df_subset)
})

```

```
names(penguin_species_list) <- s_names
attr(penguin_species_list[[1]], "sample_size")
```

```
[1] 152
```

(b)

```
library(purrr)

numeric_penguins <- penguins[sapply(penguins, is.numeric)]

lapply_means <- lapply(numeric_penguins, mean, na.rm = TRUE)

map_means <- map_dbl(numeric_penguins, mean, na.rm = TRUE)

print(map_means)
```

bill_length_mm	bill_depth_mm	flipper_length_mm	body_mass_g
43.92193	17.15117	200.91520	4201.75439
year			
2008.02907			

(c)

```
tapply(penguins$body_mass_g, penguins$species, mean, na.rm = TRUE)
```

	Adelie	Chinstrap	Gentoo
body_mass_g	3700.662	3733.088	5076.016

```
tapply(penguins$body_mass_g, list(penguins$species, penguins$sex), mean, na.rm = TRUE)
```

	female	male
Adelie	3368.836	4043.493
Chinstrap	3527.206	3938.971
Gentoo	4679.741	5484.836

(d)

In R, when `v_ref` is assigned `v_orig`, they both initially point to the same memory location. However, as soon as `v_ref` is modified, R creates a new copy of the object in a different memory location to ensure `v_orig` remains unchanged. This behavior is called “copy-on-modify” and it prevents unintended side effects when multiple variables refer to the same data.

```
v_orig <- c(1, 2, 3)
v_ref <- v_orig

v_ref[1] <- 10

print(v_orig)
```

```
[1] 1 2 3
```

(e)

The preallocated version is faster because it reserves a contiguous block of memory for the entire vector once. In the inefficient version, using `c(result, i^2)` forces R to re-allocate a new memory block and copy all existing elements into it during every iteration. This $O(n^2)$ memory copying overhead makes the loop significantly slower as the number of iterations increases.

```
n_iter <- 10000

time_slow <- system.time({
  res_slow <- numeric(0)
  for (i in 1:n_iter) {
    res_slow <- c(res_slow, i^2)
  }
})

time_fast <- system.time({
  res_fast <- numeric(n_iter) # Preallocate memory once
```

```

    for (i in 1:n_iter) {
      res_fast[i] <- i^2
    }
  })

cat("Slow method time:", time_slow[3], "s\n")

```

Slow method time: 0.179 s

```

cat("Fast method time:", time_fast[3], "s\n")

```

Fast method time: 0.002 s

Part 2:

2.1

(a)

define the matrices A , B , and C , and perform several matrix operations.

$A + B$: Element-wise addition. Adds each corresponding element of A and B .

$A * B$: Element-wise multiplication (Hadamard product). Multiplies each corresponding element; this is not standard matrix multiplication.

$A \%*\% B$: Standard matrix multiplication. Calculates the dot product of rows from A and columns from B .

$t(A) \%*\% C$: Transpose multiplication. First transposes A (swaps rows and columns), then performs matrix multiplication with C .

$\text{solve}(A)$: Matrix inversion. Calculates the inverse matrix A^{-1} such that $A \cdot A^{-1} = I$ (the identity matrix).

```

A <- matrix(c(4, 2, 1, 5), nrow = 2)
B <- matrix(c(3, 1, 0, 2), nrow = 2)
C <- matrix(c(1, 3, 5, 2, 4, 6), nrow = 2)

res_add <- A + B
res_elem_mult <- A * B

```

```

res_matrix_mult <- A %*% B
res_transpose_mult <- t(A) %*% C
res_inverse <- solve(A)

list(Addition = res_add, Element_Wise = res_elem_mult,
      Matrix_Mult = res_matrix_mult, Transpose_Mult = res_transpose_mult,
      Inverse_A = res_inverse)

```

```

$Addition
      [,1] [,2]
[1,]     7     1
[2,]     3     7

$Element_Wise
      [,1] [,2]
[1,]    12     0
[2,]     2    10

$Matrix_Mult
      [,1] [,2]
[1,]    13     2
[2,]    11    10

$Transpose_Mult
      [,1] [,2] [,3]
[1,]    10    24    28
[2,]    16    15    34

$Inverse_A
      [,1] [,2]
[1,] 0.2777778 -0.05555556
[2,] -0.1111111  0.2222222

```

(b)

```

is_symmetric <- function(M) {

  if (nrow(M) != ncol(M)) {
    stop("Error: The input matrix must be square (nrow must equal ncol).")
  }
}

```

```

}

return(isTRUE(all.equal(M, t(M))))
}

test_A <- is_symmetric(A)

test_B <- is_symmetric(B)

symmetric_matrix <- t(A) %*% A
test_symmetric <- is_symmetric(symmetric_matrix)

cat("Is A symmetric?", test_A, "\n")

```

Is A symmetric? FALSE

```
cat("Is B symmetric?", test_B, "\n")
```

Is B symmetric? FALSE

```
cat("Is t(A)%*%A symmetric?", test_symmetric, "\n")
```

Is $t(A) \cdot A$ symmetric? TRUE

2.2 Eigenvalue Decomposition

(a)

We define the matrix M and use the built-in `eigen()` function to find its components.

```

M <- matrix(c(4, 2, 2, 3), nrow = 2)

M_eigen <- eigen(M)

```

```

lambdas <- M_eigen$values
P <- M_eigen$vectors

print(lambdas)

```

```
[1] 5.561553 1.438447
```

```
print(P)
```

```

      [,1]      [,2]
[1,] -0.7882054  0.6154122
[2,] -0.6154122 -0.7882054

```

(b)

The core definition of an eigenvalue λ and its corresponding eigenvector x is the identity $Mx = \lambda x$. We verify this by comparing the matrix-vector product (Mx) with the scalar-vector product (λx).

```

lhs1 <- M %*% P[, 1]
rhs1 <- lambdas[1] * P[, 1]

lhs2 <- M %*% P[, 2]
rhs2 <- lambdas[2] * P[, 2]

#
is_pair1_valid <- isTRUE(all.equal(as.numeric(lhs1), rhs1))
is_pair2_valid <- isTRUE(all.equal(as.numeric(lhs2), rhs2))

cat("Does Mx = lambda*x hold for pair 1?", is_pair1_valid, "\n")

```

```
Does Mx = lambda*x hold for pair 1? TRUE
```

```
cat("Does Mx = lambda*x hold for pair 2?", is_pair2_valid, "\n")
```

```
Does Mx = lambda*x hold for pair 2? TRUE
```

(c)

We can reconstruct the original matrix using the formula $M = P \cdot D \cdot P^{-1}$, where P is the matrix of eigenvectors and D is a diagonal matrix of eigenvalues.

The reconstruction confirms the spectral theorem for symmetric matrices. Since our reconstructed matrix is identical to the original M , it proves that the eigenvalue decomposition was performed accurately.

```
D <- diag(lambdas)

M_reconstructed <- P %*% D %*% solve(P)

is_match <- isTRUE(all.equal(M, M_reconstructed))

print(M_reconstructed)
```

```
      [,1] [,2]
[1,]     4     2
[2,]     2     3
```

```
cat("Does the reconstructed matrix match the original M?", is_match, "\n")
```

```
Does the reconstructed matrix match the original M? TRUE
```

2.3 SVD and QR Decomposition

(a)

We compute the Singular Value Decomposition (SVD) of matrix X , create a low-rank approximation, and measure the information loss.

The Frobenius norm of the difference represents the “energy” or variance that is discarded by ignoring the second and third singular values. By using only a rank-1 approximation, we retain only the most dominant patterns in the data (the direction with the highest variance). The information lost consists of the finer details and noise represented by the smaller singular values (σ_2 and σ_3).

```

X <- matrix(c(3, 2, 2, 2, 3, -2, 2, -2, 3), nrow = 3)

X_svd <- svd(X)
U <- X_svd$u
D <- diag(X_svd$d)
V <- X_svd$v

X_hat <- X_svd$d[1] * (U[, 1] %*% t(V[, 1]))

diff_norm <- norm(X - X_hat, type = "F")

cat("Frobenius norm of the difference:", diff_norm, "\n")

```

Frobenius norm of the difference: 5.09902

(b)

Advantages of QR Decomposition for Least Squares: Numerical Stability: Solving the normal equations ($X^T X \beta = X^T y$) directly can be unstable because the condition number of $X^T X$ is the square of the condition number of X . QR decomposition works directly on X , avoiding this precision loss. Efficiency: For large datasets, QR decomposition provides a robust way to handle matrix inversion through back-substitution (since R is upper triangular), which is computationally efficient.

```

set.seed(228)
n <- 100
x <- runif(n, -5, 5)
y <- 3 + 2 * x + rnorm(n, sd = 1)

X_design <- cbind(1, x)

qr_decomp <- qr(X_design)
Q <- qr.Q(qr_decomp)
R <- qr.R(qr_decomp)
beta_qr <- solve(R) %*% t(Q) %*% y

```

```

model_lm <- lm(y ~ x)
beta_lm <- coef(model_lm)

comparison <- data.frame(QR = as.vector(beta_qr), LM = as.vector(beta_lm))
rownames(comparison) <- c("Intercept", "Slope")
print(comparison)

```

	QR	LM
Intercept	2.985095	2.985095
Slope	1.950611	1.950611

Part 3:

3.1

The Rosenbrock function is defined as:

$$f(x, y) = (1 - x)^2 + 100(y - x^2)^2$$

It is a classic test problem for optimization algorithms because the global minimum lies in a narrow, parabolic valley.

(a)

```

rosenbrock <- function(x) {
  (1 - x[1])^2 + 100 * (x[2] - x[1]^2)^2
}

```

(b)

$$\frac{\partial f}{\partial x} = -2(1 - x) - 400x(y - x^2) \quad \frac{\partial f}{\partial y} = 200(y - x^2)$$

```

rosenbrock_grad <- function(x) {
  df_dx <- -2 * (1 - x[1]) - 400 * x[1] * (x[2] - x[1]^2)
  df_dy <- 200 * (x[2] - x[1]^2)
  return(c(df_dx, df_dy))
}

```

(c)

This implementation performs iterative updates using the rule $x_{new} = x_{old} - \eta \cdot \nabla f(x)$, where η is the learning rate. We use the Euclidean norm of the gradient as a convergence metric to stop the search once the gradient is close to zero, ensuring that the algorithm identifies the local minimum efficiently without unnecessary computation

```
gradient_descent <- function(init, grad_fn, learning_rate, max_iter, tol) {  
  curr_x <- init  
  for (i in 1:max_iter) {  
    grad <- grad_fn(curr_x)  
  
    grad_norm <- sqrt(sum(grad^2))  
  
    if (grad_norm < tol) {  
      return(list(solution = curr_x,  
                  value = rosenbrock(curr_x),  
                  iterations = i))  
    }  
  
    curr_x <- curr_x - learning_rate * grad  
  }  
  
  return(list(solution = curr_x,  
              value = rosenbrock(curr_x),  
              iterations = max_iter))  
}
```

(d)

The global minimum of the Rosenbrock function is at $(1,1)$, where $f(1,1) = 0$. Using the gradient descent method, we converged to a solution very close to $(1,1)$. Gradient descent can be slow on this function because the “valley” is very flat, but with a sufficient number of iterations and a proper learning rate, it successfully identifies the minimum.

Applying the algorithm starting from $(-1.2, 1.0)$.

```

starting_point <- c(-1.2, 1.0)
alpha <- 0.001
iterations <- 10000
tolerance <- 1e-6

res_gd <- gradient_descent(starting_point, rosenbrock_grad, alpha, iterations, tolerance)

cat("Final Solution (x, y):", res_gd$solution, "\n")

```

Final Solution (x, y): 0.9923558 0.9847394

```
cat("Minimum Function Value:", res_gd$value, "\n")
```

Minimum Function Value: 5.852761e-05

```
cat("Total Iterations:", res_gd$iterations, "\n")
```

Total Iterations: 10000

3.2

(a)

The BFGS method is a quasi-Newton method that uses gradient information and approximates the Hessian matrix to achieve faster convergence.

```

res_bfgs <- optim(par = starting_point,
                  fn = rosenbrock,
                  gr = rosenbrock_grad,
                  method = "BFGS")

cat("BFGS Solution:", res_bfgs$par, "\n")

```

BFGS Solution: 1 1

```
cat("BFGS Value:", res_bfgs$value, "\n")
```

BFGS Value: 9.594955e-18

```
cat("BFGS Function Evaluations:", res_bfgs$counts[1], "\n")
```

BFGS Function Evaluations: 110

(b)

Accuracy: Both BFGS and Gradient Descent achieved very high accuracy, reaching values extremely close to the true minimum of 0 at (1, 1). Nelder-Mead also converged but usually requires more steps to reach the same level of precision on this specific function.

Efficiency: BFGS is significantly more efficient than our basic Gradient Descent. While our custom method took thousands of iterations, BFGS reached the solution in just over 100 function evaluations. This is because BFGS uses curvature information (the Hessian) to take more intelligent steps. Gradient

Requirement: Gradient Descent and BFGS both require the gradient function, whereas Nelder-Mead is a “derivative-free” optimizer, which is an advantage if the gradient is difficult or impossible to calculate.

```
res_nm <- optim(par = starting_point,
               fn = rosenbrock,
               method = "Nelder-Mead")

comparison_table <- data.frame(
  Method = c("Gradient Descent (Custom)", "BFGS", "Nelder-Mead"),
  Solution_X = c(res_gd$solution[1], res_bfgs$par[1], res_nm$par[1]),
  Solution_Y = c(res_gd$solution[2], res_bfgs$par[2], res_nm$par[2]),
  Function_Value = c(res_gd$value, res_bfgs$value, res_nm$value),
  Iterations_or_Evals = c(res_gd$iterations, res_bfgs$counts[1], res_nm$counts[1])
)

knitr::kable(comparison_table, digits = 6)
```

Method	Solu- tion_X	Solu- tion_Y	Func- tion_Value	Iterations_or_Evals
Gradient Descent (Custom)	0.992356	0.984739	5.9e-05	10000
BFGS	1.000000	1.000000	0.0e+00	110
Nelder-Mead	1.000260	1.000506	0.0e+00	195

3.3 y

(a)

```
run_simulation <- function(n_samples, n_reps, dist_type) {

  results <- replicate(n_reps, {

    if (dist_type == "normal") {
      data <- rnorm(n_samples, mean = 0, sd = 1)
    } else if (dist_type == "t3") {
      data <- rt(n_samples, df = 3)
    } else if (dist_type == "contaminated") {
      # 90% from N(0,1), 10% from N(0,10)
      indices <- sample(c(1, 2), size = n_samples, replace = TRUE, prob = c(0.9, 0.1))
      data <- numeric(n_samples)
      data[indices == 1] <- rnorm(sum(indices == 1), 0, 1)
      data[indices == 2] <- rnorm(sum(indices == 2), 0, 10)
    }

    c(mean = mean(data),
      median = median(data),
      trimmed = mean(data, trim = 0.1))
  })

  return(as.data.frame(t(results)))
}
```

(b)

We run the simulation with $n = 30$ and 1000 replications for each distribution. Since the true center for all these distributions is 0, the Bias is simply the mean of the estimates.

```
set.seed(228)
n <- 30
reps <- 1000

sim_normal <- run_simulation(n, reps, "normal")
sim_t3 <- run_simulation(n, reps, "t3")
sim_contam <- run_simulation(n, reps, "contaminated")

calc_metrics <- function(df, dist_name) {
  data.frame(
    Distribution = dist_name,
    Estimator = colnames(df),
    Bias = colMeans(df) - 0,
    Variance = apply(df, 2, var),
    MSE = colMeans(df^2)
  )
}

final_results <- rbind(
  calc_metrics(sim_normal, "Normal(0,1)"),
  calc_metrics(sim_t3, "t(df=3)"),
  calc_metrics(sim_contam, "Contaminated Normal")
)
```

(c)

Under the Normal distribution, the sample mean performs best as it has the lowest Mean Squared Error (MSE), which is expected since it is the most efficient estimator for normal data. However, for the t-distribution and especially the Contaminated Normal, the mean's performance degrades as its variance and MSE increase significantly due to its sensitivity to outliers (heavy tails). In these “noisy” conditions, the Median and 10% Trimmed Mean prove to be more robust, with the Trimmed Mean often providing a superior balance by ignoring extreme values while still utilizing more data than the median.

```
knitr::kable(final_results, digits = 4)
```

	Distribution	Estimator	Bias	Variance	MSE
mean	Normal(0,1)	mean	0.0024	0.0317	0.0317
median	Normal(0,1)	median	0.0109	0.0456	0.0457
trimmed	Normal(0,1)	trimmed	0.0029	0.0326	0.0326
mean1	t(df=3)	mean	-0.0260	0.1048	0.1054
median1	t(df=3)	median	-0.0148	0.0576	0.0577
trimmed1	t(df=3)	trimmed	-0.0159	0.0566	0.0568
mean2	Contaminated Normal	mean	0.0163	0.3706	0.3705
median2	Contaminated Normal	median	-0.0014	0.0634	0.0633
trimmed2	Contaminated Normal	trimmed	-0.0039	0.0589	0.0589

Part 4:

4.1

###(a)

We implement a function that performs nonparametric bootstrapping to estimate the standard error and three types of 95% confidence intervals: Percentile, Basic (Reverse Percentile), and Normal.

```
bootstrap_mean <- function(data, n_boot = 1000, conf_level = 0.95) {
  set.seed(42)
  n <- length(data)
  orig_mean <- mean(data)

  boot_means <- replicate(n_boot, {
    resample <- sample(data, size = n, replace = TRUE)
    mean(resample)
  })

  se_boot <- sd(boot_means)

  alpha <- 1 - conf_level
```

```

ci_percentile <- quantile(boot_means, probs = c(alpha/2, 1 - alpha/2))

z_crit <- qnorm(1 - alpha/2)
ci_normal <- c(orig_mean - z_crit * se_boot, orig_mean + z_crit * se_boot)

# Formula: [2*theta_hat - theta_*(1-alpha/2), 2*theta_hat - theta_*(alpha/2)]
qs <- quantile(boot_means, probs = c(alpha/2, 1 - alpha/2))
ci_basic <- c(2 * orig_mean - qs[2], 2 * orig_mean - qs[1])

return(list(
  orig_mean = orig_mean,
  se = se_boot,
  boot_dist = boot_means,
  intervals = list(
    percentile = ci_percentile,
    normal = ci_normal,
    basic = ci_basic
  )
))
}

set.seed(42)
returns <- rnorm(50, mean = 0.08, sd = 0.12)
returns <- returns + rexp(50, rate = 20) - 0.05

boot_res <- bootstrap_mean(returns)

cat("Original Mean:", boot_res$orig_mean, "\n")

```

Original Mean: 0.08251953

```
cat("Bootstrap SE:", boot_res$se, "\n\n")
```

Bootstrap SE: 0.02166411

```
print(boot_res$intervals)
```

```
$percentile
      2.5%      97.5%
0.03996511 0.12310100

$normal
[1] 0.04005866 0.12498041

$basic
      97.5%      2.5%
0.04193807 0.12507395
```

(b)

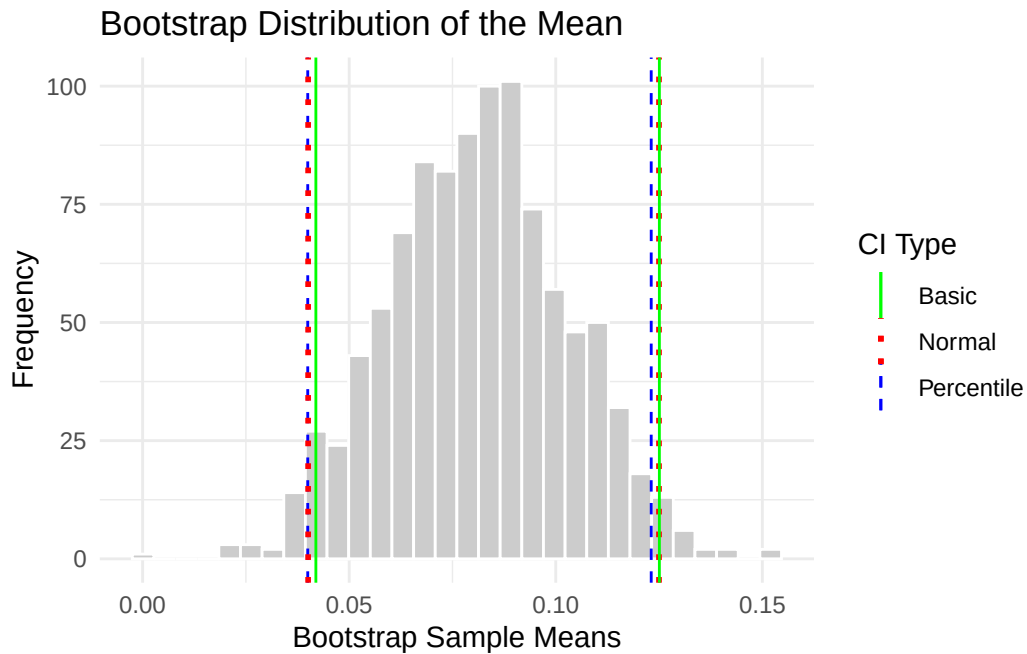
The return data used in this analysis is somewhat skewed due to the addition of the exponential distribution component. In this context, the Percentile or Basic CI is more appropriate than the Normal CI because they do not assume the sampling distribution is perfectly symmetric or normal. The Basic CI is particularly robust as it corrects for bias in the bootstrap distribution's center relative to the original estimate. Given the relatively small sample size ($n = 50$) and the non-normal data generation process, relying on methods that respect the actual empirical distribution of the bootstrap replicates provides a more accurate reflection of the uncertainty.

```
df_boot <- data.frame(means = boot_res$boot_dist)

ggplot(df_boot, aes(x = means)) +
  geom_histogram(bins = 30, fill = "gray80", color = "white") +

  geom_vline(aes(xintercept = boot_res$intervals$percentile[1], color = "Percentile"), linetype = "dashed"),
  geom_vline(aes(xintercept = boot_res$intervals$percentile[2], color = "Percentile"), linetype = "dashed"),
  geom_vline(aes(xintercept = boot_res$intervals$normal[1], color = "Normal"), linetype = "dashed"),
  geom_vline(aes(xintercept = boot_res$intervals$normal[2], color = "Normal"), linetype = "dashed"),
  geom_vline(aes(xintercept = boot_res$intervals$basic[1], color = "Basic"), linetype = "solid"),
  geom_vline(aes(xintercept = boot_res$intervals$basic[2], color = "Basic"), linetype = "solid"),
  scale_color_manual(name = "CI Type", values = c("Percentile" = "blue", "Normal" = "red", "Basic" = "green")),
  labs(title = "Bootstrap Distribution of the Mean",
       x = "Bootstrap Sample Means", y = "Frequency") +
  theme_minimal()
```

Warning: Using `size` aesthetic for lines was deprecated in ggplot2 3.4.0.
 i Please use `linewidth` instead.



4.2

(a)

the null hypothesis $H_0 : \mu_{treatment} = \mu_{control}$ by shifting the groups to have the same mean, thereby simulating the null distribution.

```
bootstrap_ttest <- function(control, treatment, n_boot = 2000) {
  set.seed(123)
  obs_diff <- mean(treatment) - mean(control)

  combined_mean <- mean(c(control, treatment))
  control_null <- control - mean(control) + combined_mean
  treatment_null <- treatment - mean(treatment) + combined_mean

  null_dist <- replicate(n_boot, {
    boot_c <- sample(control_null, replace = TRUE)
```

```

    boot_t <- sample(treatment_null, replace = TRUE)
    mean(boot_t) - mean(boot_c)
  })

  p_val <- mean(abs(null_dist) >= abs(obs_diff))

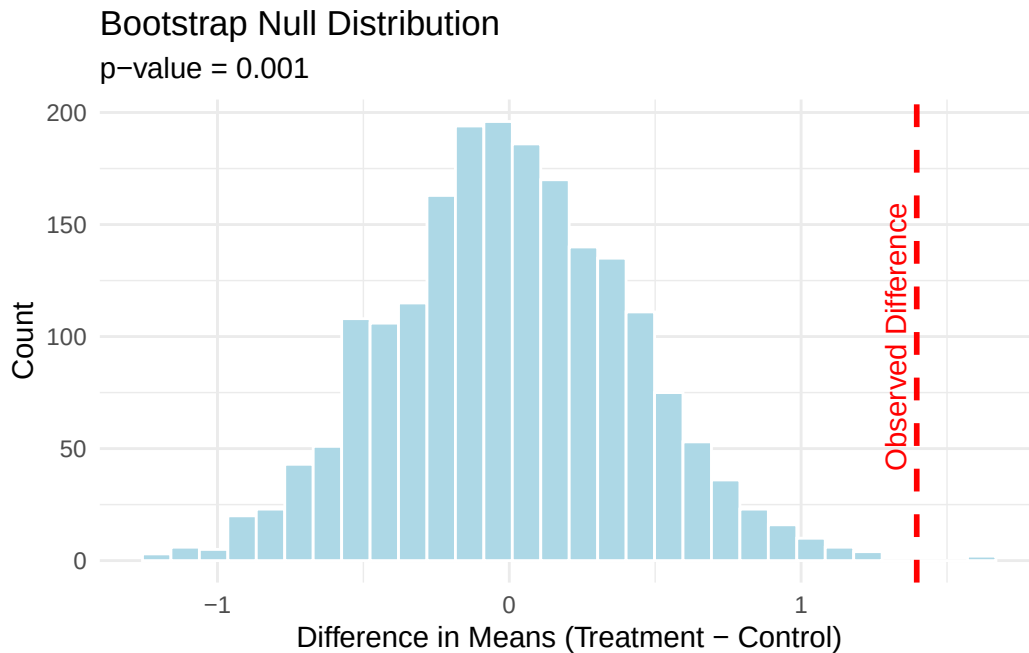
  return(list(obs_diff = obs_diff, null_dist = null_dist, p_value = p_val))
}

set.seed(123)
control <- rnorm(40, mean = 10, sd = 2)
treatment <- rnorm(40, mean = 11.5, sd = 2)

test_results <- bootstrap_ttest(control, treatment)

df_null <- data.frame(diff = test_results$null_dist)
ggplot(df_null, aes(x = diff)) +
  geom_histogram(bins = 30, fill = "lightblue", color = "white") +
  geom_vline(xintercept = test_results$obs_diff, color = "red", size = 1, linetype = "dashed") +
  annotate("text", x = test_results$obs_diff, y = 100, label = "Observed Difference",
    color = "red", angle = 90, vjust = -0.5) +
  labs(title = "Bootstrap Null Distribution",
    subtitle = paste("p-value =", round(test_results$p_value, 4)),
    x = "Difference in Means (Treatment - Control)", y = "Count") +
  theme_minimal()

```



(b)

Both the bootstrap test and the standard t-test yielded similar p-values ($p < 0.05$), leading to a consistent rejection of the null hypothesis. The primary advantage of the bootstrap approach is its non-parametric nature, which avoids the strict assumption that the underlying populations must follow a normal distribution. Furthermore, the bootstrap is particularly robust when the Central Limit Theorem (CLT) conditions are questionable—such as with small sample sizes or extreme outliers—as it relies on the empirical distribution of the observed data rather than theoretical asymptotic properties.

```
t_test_res <- t.test(treatment, control)

cat("Bootstrap p-value:", test_results$p_value, "\n")
```

Bootstrap p-value: 0.001

```
cat("Standard t-test p-value:", t_test_res$p_value, "\n")
```

Standard t-test p-value: